# AI based Release Risk Predictor

**Live repo:** https://github.com/nanantha17/

## The Problem

In large-scale software NPI programs, dozens of patches are integrated simultaneously across software teams and cross-programs. Each patch brings unknown risk:

- A single bad integration can cause production downtime
- Teams have no visibility into which patch introduced instability until it's too late
- CI/CD pipelines surface failures only after the fact — not before the decision to integrate
- Release managers are forced to make Go/NoGo decisions with incomplete, scattered signals

The result: last-minute integration failures, emergency hotfixes, delayed customer launches, and escalations that cost millions.

## The Solution

An end-to-end AI system that scores the risk brought in by each software patch before integration — giving CI/CD pipelines and release managers an early warning signal to act on.

1. Per-patch risk scoring from 13 quality signals (test coverage, defect history, CI metrics, code complexity, release notes)
2. Explainable risk factors — not just a score, but *which signals* are driving risk and by how much which is quantifiable
3. Actionable recommendations— specific steps to reduce risk before the patch is integrated
4. Ensemble ML confidence— three independent models must agree before flagging CRITICAL

Real-world impact at ASML: 40% reduction in integration issues by identifying high-risk patches early and redirecting team effort before they reached CI/CD saving millions $

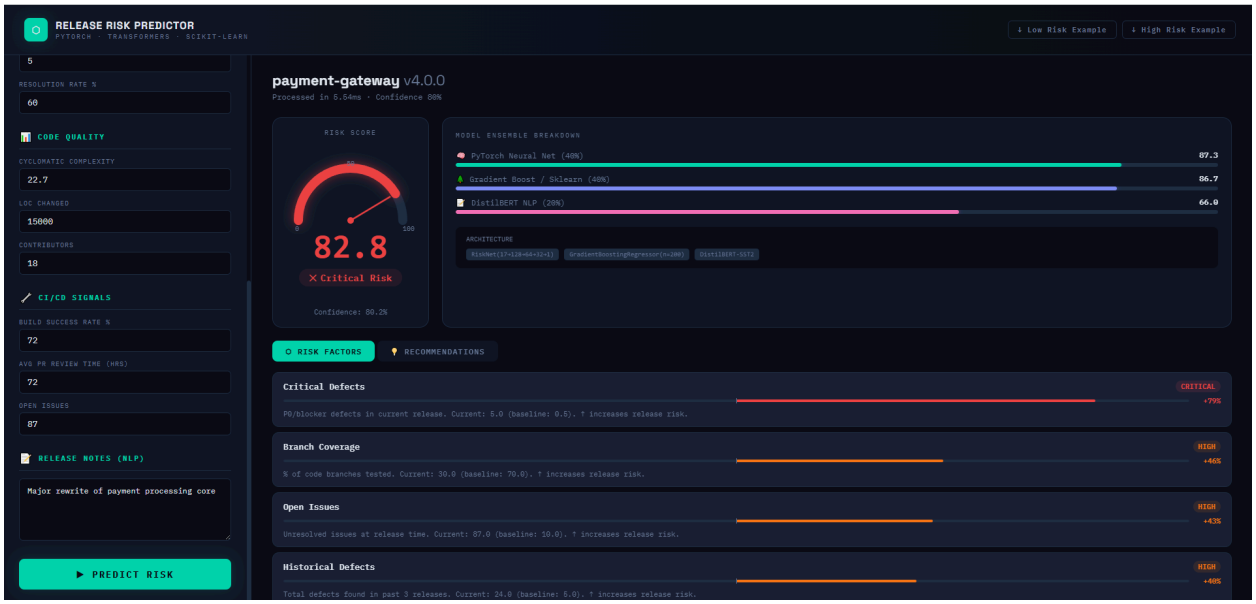
An ensemble ML system that predicts software release risk from 13 quality metrics, explains the top risk factors, and recommends actions — deployed as a REST API with an interactive React dashboard.

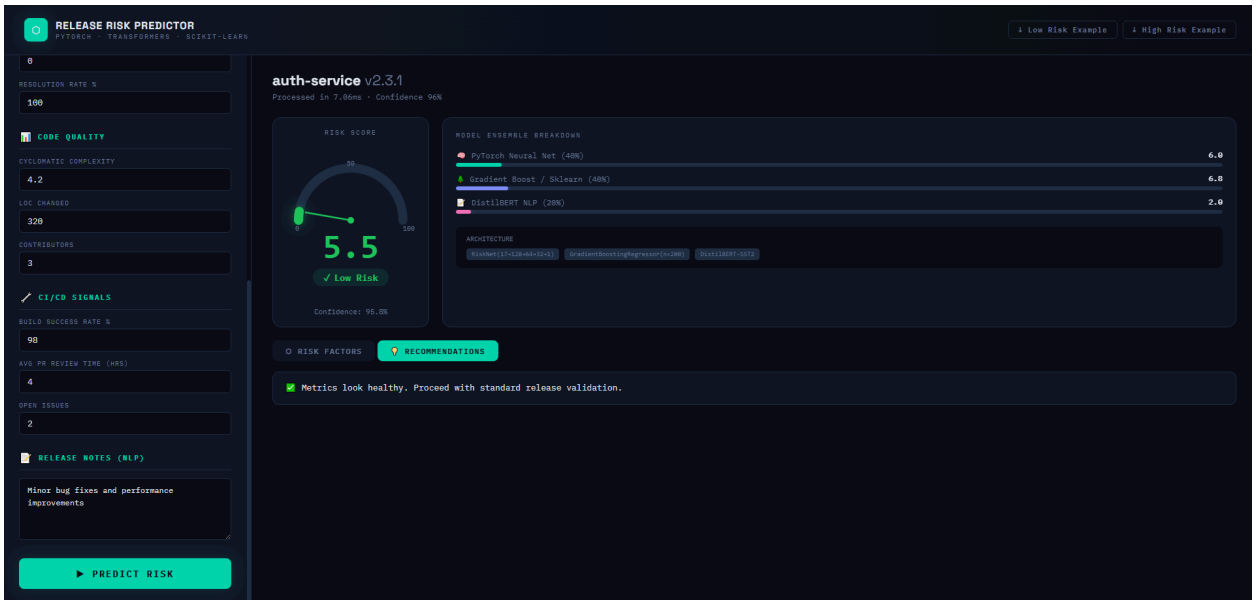
The end-to-end ML application predicts software release risk using an ensemble of:

1. PyTorch deep neural network (tabular metrics)
2. Scikit-learn Gradient Boosting Regressor

3. HuggingFace Transformers DistilBERT for release notes NLP

## Dashboard




## How It Works

If I feed it my patch/release metrics → I get back a risk score, confidence level, ranked risk factors, and actionable recommendations in under 1 second.

**Example: Catastrophic release**

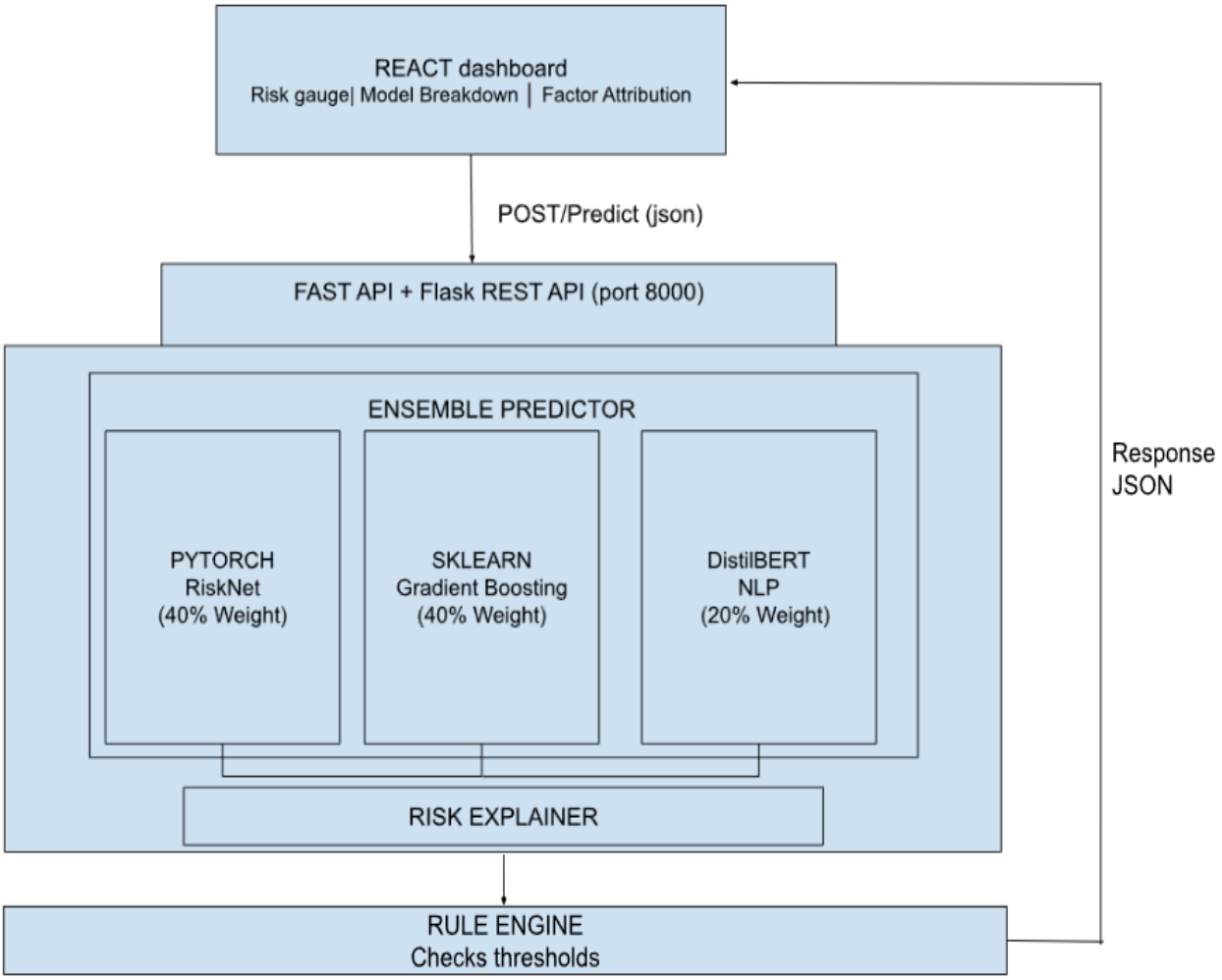
```json
{
  "risk_score": 88.4,
  "risk_level": "CRITICAL",
  "confidence": 0.97,
  "ml_breakdown": {
    "pytorch_neural_net": 87.9,
    "sklearn_gradient_boost": 87.6,
    "nlp_release_notes": 91.0
  }
}
```

**Example: Healthy release**

```json
{
  "risk_score": 5.3,
  "risk_level": "LOW",
  "confidence": 0.961,
  "ml_breakdown": {
    "pytorch_neural_net": 5.9,
    "sklearn_gradient_boost": 6.4,
    "nlp_release_notes": 2.0
  }
}
```

**83-point spread** between a catastrophic and a healthy release. All three models agree in both directions.

## Architecture




## ML Model Details

**1. PyTorch RiskNet (40% weight)**
- **Input:** 17 features (12 raw + 5 derived)
- **Architecture:** `Input(17) → BatchNorm → [256→128→64→32] → Sigmoid
  - Wider than original — better separation of extreme risk values
  - LayerNorm + GELU + Dropout at each block
- **Training:** 7,000 synthetic samples across 80 epochs
  - 6,000 base distribution samples
  - 500 deliberately extreme HIGH risk samples (forces model to learn the top end)
  - 500 deliberately extreme LOW risk samples (forces model to learn the bottom end)
- **Loss:** MSE + L1 + focal penalty on extreme samples
- **Scheduler:** CosineAnnealingLR (stepped per epoch)

- **Optimizer:**AdamW with weight decay, best-epoch checkpointing

**2. Scikit-learn GradientBoosting (40% weight)**
- `GradientBoostingRegressor(n_estimators=300, learning_rate=0.04, max_depth=5)`
- `StandardScaler` preprocessing pipeline
- Same 17-feature input as PyTorch

**3. DistilBERT NLP (20% weight)**
- `distilbert-base-uncased-finetuned-sst-2-english` via HuggingFace
- **Key fix:** DistilBERT was trained on movie reviews and misclassifies software vocabulary — words like "fix", "bug", "optimization" score as negative sentiment. The scorer uses domain-aware short-circuiting:
  - Text with clear low-risk SW keywords (`minor`, `patch`, `fix`, `docs`) → **bypass transformer, trust keywords**
  - Text with clear high-risk SW keywords (`breaking`, `emergency`, `hotfix`, `outage`) → **bypass transformer, boost score**
  - Ambiguous text → **use transformer weighted by keyword support**
- Confidence-weighted blending: transformer influence scales with its own confidence score

**Feature Engineering (17 features)**

| Feature | Type | Description |
|---|---|---|
| **test_coverage** | Raw | % of code covered by tests |
| **code_coverage** | Raw | % of lines executed |
| **branch_coverage** | Raw | % of branches tested |
| **past_defects_total** | Raw | Total defects in last 3 releases |
| **critical_defects** | Raw | Priority 1/blocker defects |
| **defect_resolution_rate** | Raw | % resolved before release |
| **cyclomatic_complexity** | Raw | Average cyclomatic complexity |
| **lines_of_code_changed** | Raw | LOC changed in this release |
| **num_contributors** | Raw | Contributing engineers |
| **build_success_rate** | Raw | CI pass rate % |
| **avg_pr_review_time_hours** | Raw | Average PR review time |
| **open_issues** | Raw | Open issues at release time |
| **coverage_avg** | Derived | Mean of test/code/branch coverage |

| | | |
|---|---|---|
| **defect_density** | Derived | Defects per KLOC changed |
| **review_burden** | Derived | LOC per contributor |
| **change_risk_index** | Derived | log1p(LOC) × complexity/10 |
| **quality_gate_score** | Derived | Weighted avg of coverage + build + resolution |

### Confidence Scoring

```
confidence = clip(1.0 - std([pytorch, sklearn, nlp]) × 2, 0.5, 0.99)
```

All three models agreeing → confidence ~0.96+. Strong disagreement → confidence floors at 0.5.

### Model Persistence

Models are saved to `saved_models/` after first training run.

- **First startup:** ~25 seconds (trains 7,000 samples × 80 epochs)
- **Subsequent startups:** ~2 seconds (loads weights from disk)
- Delete `saved_models/` to force retrain from scratch

---

## Quick Start

### Prerequisites

- Python 3.9+
- Node.js 18+

### Backend

```bash
cd backend
pip install -r requirements.txt
uvicorn main:app --reload
# API:      http://localhost:8000
# Swagger:  http://localhost:8000/docs
```

### Frontend

```bash
cd frontend
npm install
npm run dev
# Dashboard: http://localhost:3000
```

## API Reference

`POST /predict`

**Request:**

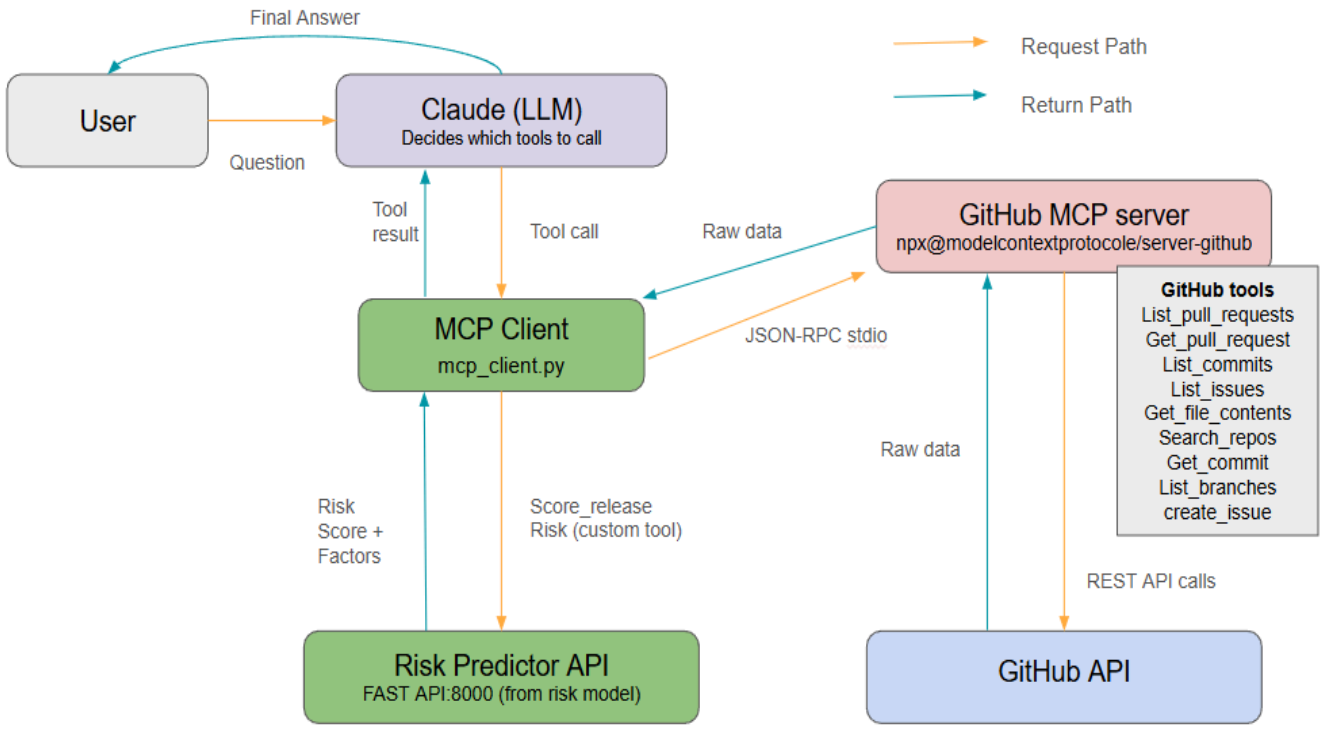
```json
{
  "package_name": "payment-gateway",
  "version": "4.0.0",
  "test_coverage": 42,
  "code_coverage": 35,
  "branch_coverage": 28,
  "past_defects_total": 20,
  "critical_defects": 5,
  "defect_resolution_rate": 50,
  "cyclomatic_complexity": 25,
  "lines_of_code_changed": 14000,
  "num_contributors": 18,
  "build_success_rate": 65,
  "avg_pr_review_time_hours": 90,
  "open_issues": 110,
  "release_notes": "Major rewrite of core payment engine, breaking changes, emergency migration required"
}
```

## Risk Level Thresholds

| Score | Level |
|---|---|
| 0 – 24 | 🟢 LOW |
| 25 – 49 | 🟡 MEDIUM |
| 50 – 74 | 🟠 HIGH |
| 75 – 100 | 🔴 CRITICAL |

## Adding Github MCP Server with separate MCP client for interfacing with Claude Agentic loop

---




1. User asks in plain language — e.g. *"What's the release risk for owner/repo?"* — typed into the CLI prompt in `mcp_client.py`
2. Claude receives the question and starts its agentic loop — reads the system prompt, sees the available tools (GitHub tools + `score_release_risk`), and decides which GitHub tools to call first to gather signals
3. MCP Client routes Claude's tool calls — `execute_tool()` in `mcp_client.py` receives each `tool_use` block and dispatches it: GitHub tools go to the GitHub MCP server subprocess, `score_release_risk` goes to the FastAPI backend
4. GitHub MCP server translates the tool call into real GitHub REST API calls — `list_pull_requests`, `list_commits`, `list_issues`, `get_commit` etc.
5. GitHub REST API returns live data — open issues, PR merge times, commit authors, lines changed, CI check statuses, issue labels (bug/critical/blocker)
6. Claude maps GitHub data to the 13 risk signals — e.g. additions+deletions → `lines_of_code_changed`, unique authors → `num_contributors`, open bug

issues → `critical_defects`, PR open→merge time → `avg_pr_review_time_hours`

7. MCP Client calls `score_release_risk` — routes to the FastAPI `/predict` endpoint built earlier, with the mapped metrics payload as JSON
8. Risk Predictor runs the ensemble — PyTorch RiskNet + Sklearn GBM + DistilBERT NLP → weighted risk score, confidence, SHAP-style factor breakdown, and recommendations returned up through the MCP client back to Claude
9. Claude synthesizes the final response — combines the risk score, top factors, and recommendations into a conversational Go/NoGo report delivered back to the user